
Improving Zero-Shot Offline RL via Behavioral Task Sampling

Nazim Bendib¹ Nicolas Perrin-Gilbert¹ Olivier Sigaud¹

Abstract

Offline zero-shot reinforcement learning (RL) aims to learn agents that optimize unseen reward functions without additional environment interaction. The standard approach to this problem trains task-conditioned policies by sampling task vectors that define linear reward functions over learned state representations. In most existing algorithms, these task vectors are randomly sampled, implicitly assuming this adequately captures the structure of the task space. We argue that doing so leads to suboptimal zero-shot generalization. To address this limitation, we propose extracting task vectors directly from the offline dataset and using them to define the task distribution used for policy training. We introduce a simple and general reward function extraction procedure that integrates into existing offline zero-shot RL algorithms. Across multiple benchmark environments and baselines, our approach improves zero-shot performance by an average of 20%, highlighting the importance of principled task sampling in offline zero-shot RL.

1. Introduction

Modern reinforcement learning (RL) agents are increasingly expected to solve not a single task, but many different tasks that are specified only at deployment time (Doncieux et al., 2018; Sigaud et al., 2023; Hughes et al., 2024). In fields such as natural language processing (Brown et al., 2020; Wei et al., 2021) and computer vision (Radford et al., 2021; Alayrac et al., 2022; Kirillov et al., 2023), this is largely achieved by training general models that can perform new tasks in a zero-shot manner when given an appropriate task specification. Achieving a similar level of flexibility in RL remains challenging, particularly when agents must operate from offline data.

¹Institut des Systèmes Intelligents et de Robotique, ISIR, Sorbonne Université, Paris, France. Correspondence to: Nazim Bendib <nazim.bendib@sorbonne-universite.fr>.

A common approach to offline zero-shot RL (ZSRL) is to learn reward-conditioned policies trained on a parametric family of reward functions. Methods such as Successor Features (SF) (Borsa et al., 2018; Touati et al., 2022) and Forward-Backward representations (FB) (Touati & Ollivier, 2021) follow this paradigm. They can be decomposed into two components: (1) a reward generation mechanism that defines a space of reward functions, and (2) a policy learning mechanism that trains a conditional policy to maximize reward functions sampled from this space. In both SF methods and FB, reward functions take the form $R_z(s) = \phi(s)^\top z$, where $\phi(s) \in \mathbb{R}^d$ is a learned state embedding, representing the state features and defining a feature space within $\mathbb{R}^d$, and z is a task vector sampled from a predefined distribution. The state representation $\phi(s)$ is intended to capture semantically meaningful properties of the state (e.g., velocity, orientation), while the task vector z specifies how these properties are weighted to define a particular task and reward function.

Prior work has primarily focused on learning rich representations $\phi(s)$ that best summarize the environment dynamics, under the assumption that richer representations induce a richer space of reward functions. In contrast, policy training is treated as a secondary component, with conditional policies trained using reward functions obtained by uniformly sampling task vectors from a unit hypersphere $\mathbb{S}^{d-1} \subset \mathbb{R}^d$.

We argue that this task sampling strategy suffers from a mismatch between the geometry of the task space and the physics of the environment. Although tasks are sampled to cover all possible directions, the environment dynamics limit the possible behaviors. As a result, most sampled tasks end up being poorly matched to the agent’s capabilities. In high-dimensional settings, a random direction is nearly orthogonal to any fixed low-dimensional subspace. Since the achievable behaviors of the agent span only a small region of the feature space, most uniformly sampled task vectors are poorly aligned with these achievable behaviors. When this happens, the reward signal becomes very weak for nearly all behaviors, making them appear similarly ineffective. This becomes hard for the agent to tell which behaviors are better, slowing down learning and leading to weak performance, not only on the sampled training tasks, but also when generalizing to new ones.

In this work, we address the challenge of suboptimal zero-shot generalization in offline reinforcement learning by moving beyond the standard practice of uniform task sampling. We demonstrate both theoretically and empirically that sampling task vectors uniformly from a high-dimensional hypersphere leads to "signal dilution", where reward variations vanish, and the learning signal becomes dominated by noise. To mitigate this, we introduce sampling from the Behavioral Task Distribution (BTD), a simple yet effective method that extracts task vectors directly from the offline dataset to ensure they reflect what is actually achievable in the environment.

Our approach is simple, method-agnostic, and can be integrated into existing offline ZSRL frameworks without modifying their representation learning objectives. Across multiple benchmark environments and baseline methods, learning the task distribution yields an average +20% increase in zero-shot performance.

Our main contributions are:

- Identifying uniform task sampling as a key bottleneck in offline zero-shot RL.
- Theoretically characterizing the resulting *signal dilution* failure mode.
- Introducing BTD-sampling, a simple plug-and-play method that replaces uniform sampling with a data-driven behavioral task distribution.
- Demonstrating an average +20% improvement in zero-shot performance across multiple environments, baselines, and task dimensions.

2. Related Work

2.1. Zero-shot Reinforcement Learning

Zero-shot reinforcement learning (ZSRL) has been studied through several families of approaches. A prominent line of work is based on successor features (SF) (Dayan, 1993; Barreto et al., 2017; Borsa et al., 2018; Chen et al., 2023), which learn universal value functions expressed as linear combinations of predefined or learned state features. These state features are typically obtained from representation learning techniques such as autoencoders (Bank et al., 2023), contrastive learning (Chen et al., 2020), low-rank decompositions (Touati et al., 2022; Jeon et al., 2024), or latent predictive models (Grill et al., 2020; Lawson et al., 2025). Extensions like the FB method (Touati & Ollivier, 2021; Touati et al., 2022) build on the SF framework while avoiding explicit learning of state features altogether. Recent work has further extended FB for policy distillation from expert data (Tirinzoni et al., 2025) and for generalization across changing dynamics (Bobrin et al., 2025). A key

advantage of SF-based methods is their low inference cost: at test time, a near-optimal policy for a new task can be obtained without additional training or planning, often via a simple regression problem. Another class of approaches focuses on learning general world models (Yu et al., 2023; Ding et al., 2024; Jiang et al., 2023), which enable ZSRL when combined with planning algorithms. Finally, some methods aim to generalize across tasks by learning latent encodings of reward functions during training (Frans et al., 2024; Ingebrand et al., 2024), typically by sampling and optimizing over randomly generated rewards. This work builds upon SF based methods and FB, which represent state-of-the-art for ZSRL due to their strong performance and minimal test-time overhead, and seeks to further enhance their performance.

2.2. Adaptive Goal Sampling

Most goal-conditioned reinforcement learning (GCRL) methods (Liu et al., 2022) assume a fixed, often uniform, distribution over goals for simplicity and benchmarking (Schaul et al., 2015; Andrychowicz et al., 2017), despite this being inefficient due to trivial, unreachable, or redundant goals. Consequently, several works move beyond uniform sampling by actively selecting or reweighting goals during training. This includes automatic curriculum (Portelas et al., 2020; Castanet et al., 2022) and autotelic methods (Colas et al., 2022) that adapt goal sampling online based on competence or learning progress (Florensa et al., 2018). Specifically, novelty-based approaches prioritize goals at the exploration frontier to discover new state-space regions (Ren et al., 2019; Campero et al., 2020), while density-based methods leverage statistical estimators to prioritize goals in sparsely visited areas of the goal space (Pitis et al., 2020; Pong et al., 2019). While those methods focus on online adaptation, in this work, we apply similar principles to the offline setting by replacing the uniform sampling of training goals with a learned behavioral task distribution. This ensures that training tasks are grounded in behavioral data, leading to more efficient learning and superior zero-shot generalization.

2.3. Inverse Reinforcement Learning

Inverse Reinforcement Learning (IRL) aims to recover reward functions from expert behavior (Ng & Russell, 2000), later evolving into the Maximum Entropy framework to handle reward ambiguity (Ziebart et al., 2008). While Deep MaxEnt IRL and Guided Cost Learning scaled these methods to high-dimensional spaces (Wulfmeier et al., 2015; Finn et al., 2016), adversarial approaches like GAIL and AIRL further refined distribution matching and reward portability (Ho & Ermon, 2016; Fu et al., 2018). Recent innovations include non-adversarial efficiency via IQ-Learn (Garg et al., 2021) and addressing complex real-world factors

like demonstrator expertise (Beliaev & Pedarsani, 2025), multi-agent dynamics (Freihaut & Ramponi, 2024), and history-dependent behavior (Ke et al., 2025). While IRL is not designed for ZSRL, our method is closely related: rather than inferring a single reward, it extracts a distribution of diverse reward functions from the offline dataset, enabling the training of a generalist agent.

3. Background

In the following, we present the problem setup and notations used throughout the paper, and review Successor Features (SF) methods (Borsa et al., 2018) and Forward–Backward (FB) representations (Touati & Ollivier, 2021).

3.1. Problem Setup and Notations

We define a Markov Decision Process (MDP) by the tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mu, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the transition probability distribution such that $\mathcal{P}(s'|s, a) = \mathbb{P}(s_{t+1} = s' | s_t = s, a_t = a)$, $\mu : \mathcal{S} \rightarrow [0, 1]$ is the initial state distribution $s_0 \sim \mu(s_0)$, and $\gamma \in [0, 1]$ is the discount factor. We operate in the offline setting, where the agent learns from a fixed dataset $\mathcal{D} = \{\tau_i\}_{i=1}^N$ consisting of trajectories collected by a single or multiple unknown behavior policies.

Let $\phi : \mathcal{S} \rightarrow \mathbb{R}^d$ denote a bounded learned state embedding. We consider linear reward functions of the form $R_z(s) = \phi(s)^\top z$, parameterized by a task vector $z \in \mathbb{S}^{d-1} \subset \mathbb{R}^d$ where $\mathbb{S}^{d-1}$ is the unit d -sphere. Let Π be the set (population) of all policies $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$, with $\Delta(\mathcal{X})$ denoting the set of all probability distributions over a set $\mathcal{X}$.

3.2. Successor Features

Given a state embedding $\phi(s) \in \mathbb{R}^d$ learned via some criteria (e.g., autoencoders or low-rank approximations), SF methods learn the successor features of a family of policies π_z for all task vectors $z \in \mathbb{R}^d$:

$$\begin{cases} \psi(s_0, a_0, z) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t \phi(s_t) \mid s_0, a_0, \pi_z \right], \\ \pi_z(s) := \arg \max_a \psi(s, a, z)^\top z \end{cases} \quad (1)$$

The successor features ψ_π satisfy the $\mathbb{R}^d$ -valued Bellman equation $\psi^\pi = \phi + \gamma P_\pi \psi^\pi$ with P_π the policy-induced transition matrix. Therefore, we can train ψ by minimizing the Bellman residual

$$\mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left\| \psi(s_t, a_t, z) - \phi(s_t) - \gamma \bar{\psi}(s_{t+1}, \pi_z(s_{t+1}), z) \right\|^2 \quad (2)$$

where $\bar{\psi}$ is a non-trainable target version of ψ , as in Deep Q-learning (Mnih et al., 2013). This objective can be improved

since we do not use the full vector $\psi(s, a, z)$; only the scalar $\psi(s, a, z)^\top z$ is required for defining the policies. Instead, we can minimize

$$\mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left(\psi(s_t, a_t, z)^\top z - \phi(s_t)^\top z - \gamma \bar{\psi}(s_{t+1}, \pi_z(s_{t+1}), z)^\top z \right)^2 \quad (3)$$

This trains $\psi(s, a, z)^\top z$ as the Q-function $Q(s, a, z)$ corresponding to the reward $R_z(s) = \phi(s)^\top z$.

Once a test reward $R_{\text{test}}(\cdot)$ is revealed, we use a set of evaluated samples $\mathcal{D}_r = \{(s_i, R_{\text{test}}(s_i))\}_{i=1}^{N_r}$ to estimate the task vector z_{test} . Specifically, we solve

$$z_{\text{test}} := \arg \min_z \mathbb{E}_{s \sim \mathcal{D}_r} \left[(R_{\text{test}}(s) - \phi(s)^\top z)^2 \right] \quad (4)$$

$$= \mathbb{E}_{\mathcal{D}_r} [\phi \phi^\top]^{-1} \mathbb{E}_r [\phi R_{\text{test}}], \quad (5)$$

and define the policy

$$\pi_{z_{\text{test}}}(s) = \arg \max_{a \in \mathcal{A}} \psi(s, a, z_{\text{test}})^\top z_{\text{test}}.$$

This policy is guaranteed to be optimal for all rewards in the linear span of the features ϕ .

3.3. Forward Backward

Let M^π be the successor measure defined as:

$$M^\pi(s_0, a_0, s) := \mathbb{E} \left[\sum_{t \geq 0} \gamma^t \mathbb{I}\{s_t = s\} \mid s_0, a_0, \pi \right]. \quad (6)$$

The goal of FB is to learn representations $F : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $B : \mathcal{S} \rightarrow \mathbb{R}^d$ to learn a finite-rank model of the measure M^π such that:

$$\begin{cases} \psi(s_0, a_0, z) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t \phi(s_t) \mid s_0, a_0, \pi_z \right], \\ \pi_z(s) := \arg \max_a \psi(s, a, z)^\top z \end{cases}$$

The successor measure M^π satisfies a Bellman-like equation $M^\pi = P + \gamma P_\pi M^\pi$. Therefore, we can train F and B by minimizing the Bellman residual on the parametric model $M = F^\top B$. This results in minimizing:

$$\begin{aligned} & \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left(F(s_t, a_t, z)^\top B(s') \right. \\ & \quad \left. - \gamma \bar{F}(s_{t+1}, \pi_z(s_{t+1}), z)^\top \bar{B}(s') \right)^2 \\ & \quad - 2 \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[F(s_t, a_t, z)^\top B(s_{t+1}) \right] \end{aligned} \quad (7)$$

where $\bar{F}$ and $\bar{B}$ are non-trainable target versions of F and B . Similarly to SF above, only $F(s, a, z)^\top z$ is needed to

define the policy. So we include an auxiliary loss to focus training on $F(., z)^\top z$:

$$\mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left(F(s_t, a_t, z)^\top z - B(s_{t+1})^\top (\mathbb{E}_\rho B B^\top)^{-1} z - \gamma \bar{F}(s_{t+1}, \pi_z(s_{t+1}), z)^\top z \right)^2. \quad (8)$$

This trains $F(s, a, z)^\top z$ as the Q-function $Q(s, a, z)$ to the reward $R_z(s) = B(s)^\top \mathbb{E}[B B^\top]^{-1} z$.

Once a test reward function $R_{\text{test}}(\cdot)$ is revealed, we estimate

$$z_{\text{test}} := \mathbb{E}_{s \sim \mathcal{D}_r} [R_{\text{test}} B(s)]. \quad (9)$$

and similarly define the policy

$$\pi_{z_{\text{test}}}(s) = \arg \max_{a \in \mathcal{A}} F(s, a, z_{\text{test}})^\top z_{\text{test}}.$$

This policy is guaranteed to be optimal for all rewards in the linear span of the features ϕ .

3.4. Connections between SF and FB

FB representations are related to SF, by setting $\psi(s, a, z) := F(s, a, z)$ and $\phi(s) = \mathbb{E}[B B^\top]^{-1} B(s)$. Thus, FB can be used to produce both ϕ and ψ in SF, although training is different: SF methods require a trained ϕ to train a policy while FB learns ϕ simultaneously with the policy.

In what follows, to view SF and FB as instances of the same framework, we standardize the notation by using $\phi(s)$ to denote the learned state embeddings in both methods. This allows a unified treatment of SF and FB representations.

4. Method

We define the **feature occupancy** $\psi^\pi \in \mathbb{R}^d$ of a policy π for a given MDP $\mathcal{M}$ as the discounted feature expectation:

$$\psi^\pi := \mathbb{E}_{s_0 \sim \mu} \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) \mid \pi \right]. \quad (10)$$

The feature occupancy ψ^π compresses the behavior of policy π into a single vector that summarizes which state features are induced following that policy. While $\phi(s)$ encodes state features (e.g., velocity, contact), ψ^π captures their total discounted accumulation along trajectories generated by π , i.e., the region of the feature space that the policy occupies under the MDP dynamics. In particular, for linear rewards of the form $R_z(s) = \phi(s)^\top z$, the expected discounted return

of policy π for a reward function $R_z(\cdot)$ can be written as

$$J(\pi, z) = \mathbb{E}_{s_0 \sim \mu} \left[\sum_{t=0}^{\infty} \gamma^t R_z(s_t) \mid \pi \right] \quad (11)$$

$$= \mathbb{E}_{s_0 \sim \mu} \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t)^\top z \mid \pi \right] \quad (12)$$

$$= (\psi^\pi)^\top z. \quad (13)$$

Note that $J(\pi, z) = \|\psi^\pi\|_2 \cos(\psi^\pi, z)$ for $z \in \mathbb{S}^{d-1}$, which measures the projection of the feature occupancy induced by policy π onto the task direction z , jointly reflecting alignment and magnitude along z .

Note that feature occupancy ψ^π is different from successor features $\psi(s, a, z)$ because feature occupancy is an unconditional expectation over the policy's stationary distribution, while successor features are conditional expectations that depend on a specific starting state and action.

We denote by Ψ the **behavioral space** of the MDP $\mathcal{M}$, which is the set of all feature occupancies achievable by any policy $\pi \in \Pi$:

$$\Psi = \{\psi^\pi \in \mathbb{R}^d \mid \pi \in \Pi\}. \quad (14)$$

Since $\phi(s)$ is bounded, each feature occupancy ψ^π is a discounted sum of bounded vectors, so there exists C_Ψ with $\|\psi^\pi\|_2 \leq C_\Psi$ for all $\pi \in \Pi$, making Ψ a bounded set in $\mathbb{R}^d$.

4.1. Limits of Uniform Sampling

Since we are training a task-conditioned policy, the goal is to learn $\pi^*(\cdot|z) = \arg \max_{\pi \in \Pi} J(\pi, z)$, $\forall z \in \mathbb{S}^{d-1}$. A critical requirement for learning π^* is that the expected return $J(\pi, z)$ must vary sufficiently over the policy space to distinguish better policies from worse ones. We measure this variation as the variance of expected returns across different policies for a given task. We show that uniform task sampling in high dimensions makes this performance variation vanish.

Proposition 4.1 (Vanishing Task-Specific Variance). *Let $z \sim \text{Unif}(\mathbb{S}^{d-1})$ be a task vector. Let $\Psi \subset \mathbb{R}^d$ be the behavioral space, and let Σ_Ψ denote its covariance matrix. Let $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d \geq 0$ be the eigenvalues of Σ_Ψ . Then the expected variance of the returns across the policy population satisfies:*

$$\mathbb{E}_{z \sim \text{Unif}(\mathbb{S}^{d-1})} [\text{Var}_{\pi \sim \Pi} (J(\pi, z))] = \frac{1}{d} \sum_{i=1}^d \lambda_i, \quad (15)$$

and converges to 0 as $d \rightarrow \infty$.

For proof see Appendix A.

In Proposition 4.1, the quantity $\text{Var}_{\pi \sim \Pi}(J(\pi, z))$ represents the variance of expected returns across the entire policy population Π for a fixed task z , not the variance in return estimation for a single policy. For uniform task sampling, this inter-policy variance is reduced as the ambient dimension d grows. Consequently, the gap in expected return between any two policies π_1 and π_2 for a task $z \sim \text{Unif}(\mathbb{S}^{d-1})$ shrinks toward zero.

In practice, this vanishing return signal yields gradient signals of very small magnitude, likely to be dominated by the training noise preventing training from reliably identifying the optimal behaviors. This degrades the generalization capability of the policy, resulting in suboptimal performance on training tasks and even poorer performance on unseen test tasks.

4.2. Dataset-driven task vectors

Rather than sampling task vectors uniformly from the unit hypersphere, we propose constructing tasks directly from the offline dataset. Concretely, given an offline dataset $\mathcal{D}$, we note $\mathcal{D}_{\text{sub}}$ the set of all possible contiguous subtrajectories of consecutive transitions extracted from $\mathcal{D}$.

We define p_{data} as the empirical set of task vectors extracted from the offline dataset:

$$p_{\text{data}} = \{z_\tau \mid \tau \in \mathcal{D}_{\text{sub}}\}, \quad (16)$$

with:

$$z_\tau = \frac{\tilde{\psi}^\tau}{\|\tilde{\psi}^\tau\|_2}, \quad \tilde{\psi}^\tau = \sum_{t=0}^{|\tau|-1} \gamma^t \phi(s_t), \quad (17)$$

where $\tilde{\psi}^\tau$ is the empirical feature occupancy induced by the trajectory τ . Accordingly, z_τ defines the task vector that maximizes the reward value assigned to trajectory τ , i.e., $z_\tau = \arg \max_z (\tilde{\psi}^\tau)^\top z$. Note that this does not imply that τ is an optimal trajectory for R_{z_τ} ; rather, since τ is generated by a behavioral policy $\pi_\beta \in \Pi$ (in the data collection process), the empirical feature occupancy $\tilde{\psi}^\tau$ approximates the true feature occupancy of π_β , which lies in the behavioral space Ψ .

By deriving tasks from the offline dataset, we sample task vectors z_τ that correspond to actual behaviors observed in the environment. These vectors naturally align with the directions in which policies can produce meaningful variation of the return. In contrast, uniformly sampled tasks are equally likely to point in any direction, most of which are orthogonal or poorly aligned with the achievable behaviors. As shown in Proposition 4.1, this leads to vanishing variance of the return as the feature dimension increases, making it difficult for the agent to distinguish better policies from worse ones. Sampling from p_{data} avoids this issue by concentrating on inherently meaningful tasks, allowing the

policy to reliably identify optimal behaviors. This creates a more informative learning signal throughout training, which stabilizes policy optimization and ultimately leads to better zero-shot generalization to unseen tasks.

Importantly, this approach is well-aligned with how test tasks are typically designed in practice. In common RL benchmarks, test tasks are physically grounded variations of behavior within the environment dynamics (e.g., different locomotion gaits, jumps, or flips). Although these tasks are not seen during training, they are not arbitrary, as they encourage behaviors that are plausible under the physics of the environment and are therefore likely to be represented within the behavioral space Ψ .

4.3. Learning a Behavioral Task Distribution

In practice, to capture the distribution of achievable tasks, we fit a parametric density model $p_\theta(z)$ to the empirical task set p_{data} . We refer to this fitted distribution as the Behavioral Task Distribution (BTD). To construct p_{data} , we sample N_τ subtrajectories of random lengths from the dataset, and extract task vectors from these subtrajectories as described in Section 4.2. In our implementation, p_θ is a Gaussian Mixture Model trained by maximum likelihood on p_{data} .

During policy training, we replace uniform sampling of task vectors from the unit hypersphere with sampling from the learned BTD, $z \sim p_\theta(z)$. All other components of the offline ZSRL algorithm remain unchanged. This modification biases training towards tasks that fall within the behavioral space, while preserving compatibility with existing representation learning and policy optimization pipelines. All implementation details about our method are presented in Appendix B.

5. Experiments

In this section, we aim to evaluate the proposed BTD-sampling method across multiple aspects of ZSRL. Our experiments are designed to answer four questions: **(1) Overall zero-shot performance:** How well do policies trained with BTD-sampling perform compared to uniform task sampling across benchmark environments? **(2) Robustness to representation methods:** Does BTD-sampling improve performance consistently across different latent encodings? **(3) Robustness of BTD to latent dimension:** Does BTD-sampling continue to perform well as the latent dimension grows? **(4) Combining task sampling strategies:** Can blending BTD-sampling with baseline task sampling lead to further improvements?

Environments: We conduct experiments on standard ZSRL benchmarks from ExoRL (Yarats et al., 2022), including Cheetah (run backward, walk backward, walk forward,

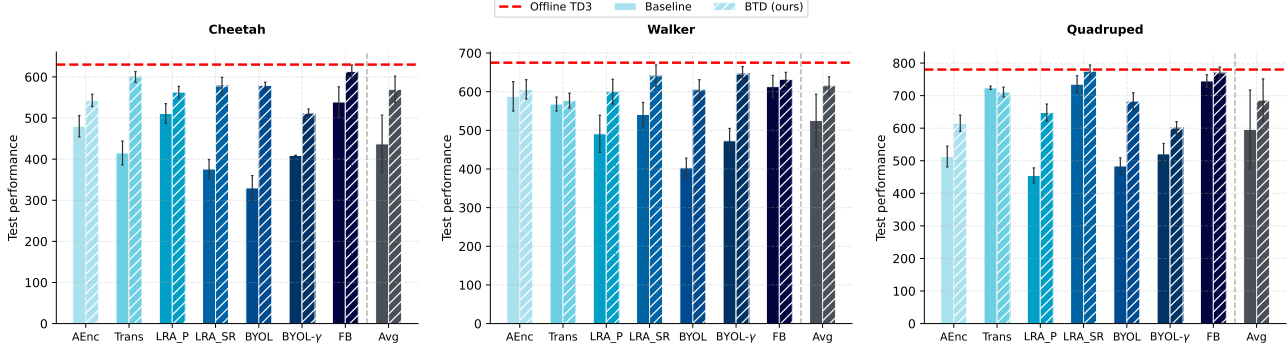


Figure 1. **Zero-shot performance comparison across task sampling strategies.** Results on Cheetah, Walker, and Quadraped for multiple representations show that BTB-sampling improves average performance and reduces sensitivity to representation choice. The horizontal red line indicates offline TD3 (oracle).

run forward), Walker (stand, walk, run, flip), and Quadraped (stand, walk, run, jump). For the main results, policies are trained using datasets collected by Random Network Distillation (Burda et al., 2018). Further details about the tasks are provided in Appendix C. Performance for each environment is computed as the average over its test tasks.

Baselines: We consider seven baselines:

- **Reconstruction and dynamics-based methods:** Autoencoder (AEnc) (Chen & Guo, 2023), Transition model (Trans), Lower-Rank Approximation of the transition probability (LRA_P) and of the successor measure (LRA_SR) (Touati et al., 2022), to learn state encoder $\phi(s)$, on top of which we train successor features.
- **Self-supervised representation learning methods:** BYOL (Grill et al., 2020) and BYOL- γ (Lawson et al., 2025) learn the state encoder $\phi(s)$ through latent-predictive learning, which we then use for learning successor features.
- **Forward-backward representations (FB)** (Touati & Olivier, 2021).

For more details about the baselines, see Appendix D.

For each baseline, we compare two main variants that differ only in how reward functions are sampled during policy training: the original method using uniformly sampled task vectors (**Baseline**) and our proposed BTB-sampling variant (**BTB**).

Implementation: Across all experiments, we first train the state encoder $\phi(s)$, then we fix it and train the policy. Policy learning is performed using offline TD3 (Fujimoto et al., 2018). Network architectures and training hyperparameters are kept fixed across methods to isolate the effect of the task sampling strategy (see Appendix B.4). The latent task dimension is set to 50 by default, unless specified otherwise. The performance for one environment is computed as

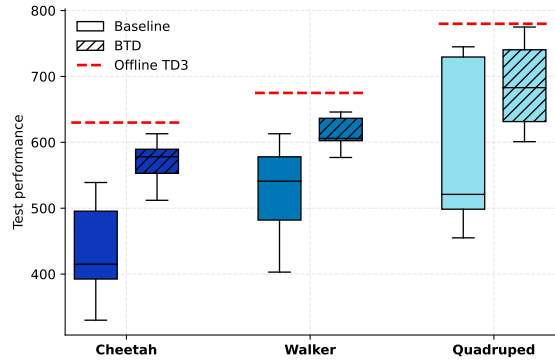


Figure 2. **Average test performance and variability of all baseline methods (solid) against their BTB-sampling counterparts (hatched) across three environments.** BTB-sampling consistently improves performance while significantly reducing variance, raising the lower bound of performance toward the offline TD3 oracle.

the average over its test tasks. For each test task, to estimate the corresponding task vector, we reveal the rewards of 5120 random states as in (Touati et al., 2022). All reported results are averaged over 5 seeds.

5.1. Overall Zero-Shot Performance

To assess the effectiveness of BTB-sampling for ZSRL, we compare baseline methods with their BTB-sampling counterparts across multiple environments and tasks.

Results As illustrated in Figure 1, our proposed BTB-sampling method consistently outperforms the corresponding baselines in nearly all experimental settings, achieving superior test performance in 13 of 15 tests and comparable performance in the remaining two, with an average improvement of +20%. This gain is particularly substantial in the Cheetah environment and when using AEnc and LRA_P to learn state embeddings. Moreover, BTB-sampling improves

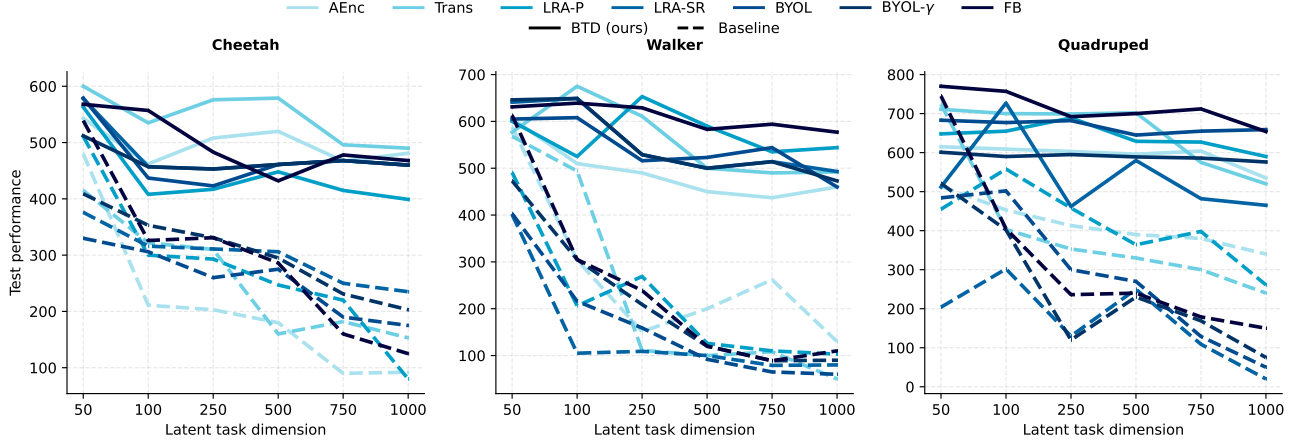


Figure 3. Test performance across a range of latent task dimensions d . Baseline performance drops in high dimensions as task vectors become increasingly orthogonal to the behavioral space, diluting the training signal. In contrast, BTB-sampling maintains robust performance by learning a task distribution that stays within the behavioral space, ensuring a consistent and informative training signal.

upon the state-of-the-art **FB** across all three environments. In general, these results show that BTB-sampling reliably improves performance in various locomotion tasks using various baseline methods.

5.2. Sensitivity to Representation Learning Methods

To evaluate the robustness of BTB-sampling across all baseline methods, we compute their average performance for each environment and compare the resulting distributions between the baseline and BTB-sampling, providing a measure of both overall performance and variability.

Results As illustrated in Figure 2, BTB-sampling consistently improves test performance while reducing the variance in all environments. The resulting test performance distributions are significantly more compact, indicating a higher degree of consistency compared to the baseline. Specifically, BTB-sampling significantly raises the lower bound of performance and improves all methods, including low-performing ones.

5.3. Effect of task space dimension

Proposition 4.1 predicts that baseline methods are increasingly likely to suffer from signal dilution as the latent task dimension grows, resulting in degraded performance. We evaluate this empirically and investigate whether BTB-sampling can maintain robust performance under the same conditions. To do so, we train policies using both baseline methods and BTB-sampling across a range of latent dimensions [50, 100, 250, 500, 750, 1000] and measure their performance in test tasks. This setup enables a direct comparison of the sensitivity of baselines to high-dimensional task spaces with the robustness offered by BTB-sampling.

Results Figure 3 shows that, consistent with Proposition 4.1, the baselines exhibit a sharp performance degradation as the dimension of the task space increases. In all three environments, the baselines struggle in high-dimensional settings, with scores often dropping rapidly as the dimension grows beyond 100. Particularly, in the Cheetah and Walker tasks, the performance of baselines collapses to near-failure performance at dimensions 750 and 1000.

This trend empirically confirms that uniform sampling approaches are highly sensitive to the dimensionality of the task space. As the dimension grows, uniformly sampled task vectors become increasingly likely to be orthogonal to the behavioral space, resulting in rewards that fail to discriminate between different behaviors.

In contrast, BTB-sampling demonstrates remarkable robustness to high-dimensional task spaces. While the performance of baselines decays rapidly, BTB-sampling maintains relatively stable and superior performance across the entire spectrum of latent dimensions. Even at dimensions as high as 1000, BTB-sampling models retain significantly higher scores compared to their baseline counterparts. These results indicate that BTB-sampling effectively mitigates the adverse effects of high-dimensional spaces. It does so by leveraging the offline dataset to align the task distribution with the behavioral space. This stabilizes the underlying baseline methods against the performance drops predicted by our theoretical analysis.

5.4. Combining task sampling strategies

We investigate whether combining BTB-sampling with existing task sampling strategies can further improve zero-shot performance. Specifically, we train policies using mixtures of tasks sampled from BTB and from a uniform distribution

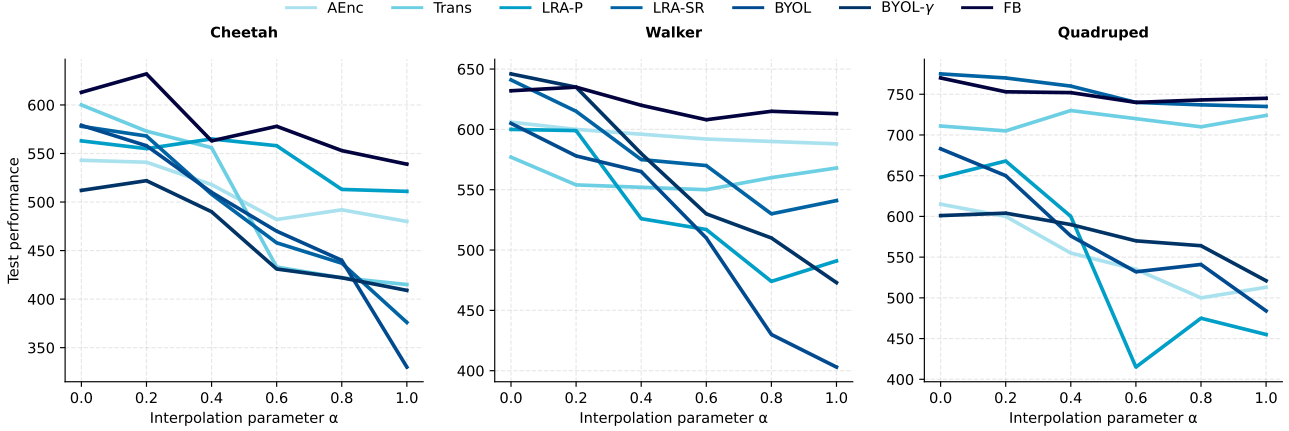


Figure 4. Test performance across Cheetah, Walker, and Quadraped as the interpolation parameter α varies from 0 (pure BTD-sampling) to 1 (pure uniform). Increasing the proportion of uniformly sampled tasks leads to performance degradation across almost all methods. This shows that uniform sampling dilutes the training signal rather than enhancing generalization. This confirms that aligning the task distribution with the behavioral space is essential for effective zero-shot transfer.

over the unit sphere $\text{Unif}(\mathbb{S}^{d-1})$. By varying the mixture proportion, we aim to understand whether hybrid task sampling strategies can yield additional gains in generalization, or whether the introduction of uniformly random tasks interferes with the effectiveness of BTD-guided exploration.

Results Figure 4 shows that performance consistently degrades as the interpolation parameter α increases. This trend is particularly pronounced in Cheetah and Walker, where performance drops across all tasks. In Quadraped, the effect is less evident, mainly because the baseline performance is already comparable to that of the BTD-sampling policy. Overall, these results indicate that incorporating uniformly random tasks does not improve robustness and instead harms policy performance. This underscores the importance of aligning the task distribution with the behavioral space to enable effective zero-shot generalization.

5.5. Ablation study

We validate key components in Appendix E: (1) comparisons of BTD-sampling to alternative sampling heuristics to determine whether the performance of BTD comes from the continuity of the distribution or from the dataset alone; (2) empirical verification of signal dilution, as demonstrated in Proposition 4.1; (3) effect of the number of GMM components on zero-shot performance; (4) effect of the number of subtrajectories N_τ on GMM estimation quality; and (5) visualization of the task space to assess how effective uniform sampling is at recovering behaviorally grounded tasks.

6. Conclusion

In this work, we identified a flaw in current offline zero-shot reinforcement learning (ZSRL): the reliance on uniform task sampling from the unit hypersphere during policy train-

ing. Our theoretical analysis reveals a “signal dilution” phenomenon in which reward variance vanishes as task dimensions increase, causing the learning signal to collapse. Crucially, we observed that the task distributions derived from the offline dataset is less affected by this growth, providing a robust signal regardless of the ambient dimension.

To back up this observation, we introduced the Behavioral Task Distribution (BTD) sampling, which extracts relevant task vectors from offline data to guide policy training. By grounding the task distribution in achievable behavioral spaces, BTB-sampling ensures that the learning process remains informative at any scale. Our empirical results demonstrate that BTB-sampling consistently outperforms standard baselines, achieving an average 20% improvement in zero-shot performance in the kind of physically relevant tasks one actually cares about in real-world deployment. Ultimately, our findings highlight that in offline ZSRL, defining which tasks to learn is as important as learning how to solve them, and that building on data-driven task distributions offers a scalable path toward more generalizable agents.

Limitations. BTB is bounded by the behavioral support of the offline dataset: task vectors are extracted from observed trajectories, so feasible but rarely observed behaviors may be underrepresented in the learned distribution. Additionally, BTB inherits the linear reward assumption of SF/FB methods, as rewards outside the linear span of ϕ cannot be recovered regardless of the task sampling strategy. Finally, our experiments focus on locomotion tasks from the ExORL benchmark. Generalization to visually complex or contact-rich manipulation environments remains an open direction.

Impact Statement

Our method operates entirely on pre-collected offline datasets and does not introduce new capabilities for harmful applications beyond those of existing offline RL methods. We do not anticipate significant negative societal impacts from this research.

Acknowledgments

Experiments presented in this paper were carried out using the HPC resources of IDRIS under the allocation 2025-[AD011016374] made by GENCI. This work was supported by the Sorbonne Center for Artificial Intelligence (SCAI).

References

- Alayrac, J.-B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., and Zaremba, W. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.
- Bank, D., Koenigstein, N., and Giryes, R. Autoencoders. *Machine learning for data science handbook: data mining and knowledge discovery handbook*, pp. 353–374, 2023.
- Barreto, A., Dabney, W., Munos, R., Hunt, J. J., Schaul, T., van Hasselt, H. P., and Silver, D. Successor features for transfer in reinforcement learning. *Advances in neural information processing systems*, 30, 2017.
- Beliaev, M. and Pedarsani, R. Inverse reinforcement learning by estimating expertise of demonstrators. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025. URL <https://doi.org/10.48550/arxiv.2402.01886>.
- Bobrin, M., Zisman, I., Nikulin, A., Kurenkov, V., and Dylov, D. Zero-shot adaptation of behavioral foundation models to unseen dynamics. *arXiv preprint arXiv:2505.13150*, 2025.
- Borsa, D., Barreto, A., Quan, J., Mankowitz, D., Munos, R., Van Hasselt, H., Silver, D., and Schaul, T. Universal successor features approximators. *arXiv preprint arXiv:1812.07626*, 2018.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Burda, Y., Edwards, H., Storkey, A., and Klimov, O. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018.
- Campero, A., Raileanu, R., Küttler, H., Tenenbaum, J. B., Rocktäschel, T., and Grefenstette, E. Learning with amigo: Adversarially motivated intrinsic goals. *arXiv preprint arXiv:2006.12122*, 2020.
- Castanet, N., Lamprier, S., and Sigaud, O. Stein variational goal generation for adaptive exploration in multi-goal reinforcement learning. *arXiv preprint arXiv:2206.06719*, 2022.
- Chen, B., Zhu, C., Agrawal, P., Zhang, K., and Gupta, A. Self-supervised reinforcement learning that transfers using random features. *Advances in Neural Information Processing Systems*, 36:56411–56436, 2023.
- Chen, S. and Guo, W. Auto-encoders in deep learning—a review with new perspectives. *Mathematics*, 11(8):1777, 2023.
- Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pp. 1597–1607. PmLR, 2020.
- Colas, C., Karch, T., Sigaud, O., and Oudeyer, P.-Y. Autotelic agents with intrinsically motivated goal-conditioned reinforcement learning: a short survey. *Journal of Artificial Intelligence Research*, 74:1159–1199, 2022.
- Dayan, P. Improving generalization for temporal difference learning: The successor representation. *Neural computation*, 5(4):613–624, 1993.
- Ding, Z., Zhang, A., Tian, Y., and Zheng, Q. Diffusion world model. *CoRR*, 2024.
- Doncieux, S., Filliat, D., Díaz-Rodríguez, N., Hospedales, T., Duro, R., Coninx, A., Roijers, D. M., Girard, B., Perrin, N., and Sigaud, O. Open-ended learning: a conceptual framework based on representational redescription. *Frontiers in neurorobotics*, 12:59, 2018.
- Finn, C., Levine, S., and Abbeel, P. Guided cost learning: Deep inverse optimal control via policy optimization. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pp. 49–58, 2016.
- Florensa, C., Held, D., Geng, X., and Abbeel, P. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pp. 1515–1528. PMLR, 2018.

- Frans, K., Park, S., Abbeel, P., and Levine, S. Unsupervised zero-shot reinforcement learning via functional reward encodings. *arXiv preprint arXiv:2402.17135*, 2024.
- Freihaut, T. and Ramponi, G. On feasible rewards in multi-agent inverse reinforcement learning. *arXiv preprint arXiv:2411.15046*, 2024. URL <https://doi.org/10.48550/arxiv.2411.15046>.
- Fu, J., Levine, S., and Abbeel, P. Learning robust rewards with adversarial inverse reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2018.
- Fujimoto, S., Hoof, H., and Meger, D. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, pp. 1587–1596. PMLR, 2018.
- Garg, D., Chakraborty, S., Cundy, C., Song, J., and Ermon, S. Iq-learn: Inverse soft-q learning for imitation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pp. 29328–29339, 2021.
- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P., Buchatskaya, E., Doersch, C., Avila Pires, B., Guo, Z., Gheshlaghi Azar, M., et al. Bootstrap your own latent—a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284, 2020.
- Ho, J. and Ermon, S. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4565–4573, 2016.
- Hughes, E., Dennis, M., Parker-Holder, J., Behbahani, F., Mavalankar, A., Shi, Y., Schaul, T., and Rocktaschel, T. Open-endedness is essential for artificial superhuman intelligence. *arXiv preprint arXiv:2406.04268*, 2024.
- Ingebrand, T., Zhang, A., and Topcu, U. Zero-shot reinforcement learning via function encoders. *arXiv preprint arXiv:2401.17173*, 2024.
- Jeen, S., Bewley, T., and Cullen, J. Zero-shot reinforcement learning from low quality data. *Advances in Neural Information Processing Systems*, 37:16894–16942, 2024.
- Jiang, Z., Xu, Y., Wagener, N., Luo, Y., Janner, M., Grefenstette, E., Rocktäschel, T., and Tian, Y. H-gap: Humanoid control with a generalist planner. *arXiv preprint arXiv:2312.02682*, 2023.
- Ke, J., Wu, F., Wang, J., Markowitz, J., and Wu, A. Inverse reinforcement learning with switching rewards and history dependency for characterizing animal behaviors. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://doi.org/10.48550/arxiv.2501.12633>.
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A. C., Lo, W.-Y., et al. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4015–4026, 2023.
- Lawson, D., Hugessen, A., Cloutier, C., Berseth, G., and Khetarpal, K. Self-predictive representations for combinatorial generalization in behavioral cloning, 2025. URL <https://arxiv.org/abs/2506.10137>.
- Liu, M., Zhu, M., and Zhang, W. Goal-conditioned reinforcement learning: Problems and solutions. *arXiv preprint arXiv:2201.08299*, 2022.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- Ng, A. Y. and Russell, S. J. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning (ICML)*, pp. 663–670, 2000.
- Pitis, S., Chan, H., Zhao, S., Stadie, B., and Ba, J. Maximum entropy gain exploration for long horizon multi-goal reinforcement learning. In *International conference on machine learning*, pp. 7750–7761. PMLR, 2020.
- Pong, V. H., Dalal, M., Lin, S., Nair, A., Bahl, S., and Levine, S. Skew-fit: State-covering self-supervised reinforcement learning. *arXiv preprint arXiv:1903.03698*, 2019.
- Portelas, R., Colas, C., Weng, L., Hofmann, K., and Oudeyer, P.-Y. Automatic curriculum learning for deep rl: A short survey. *arXiv preprint arXiv:2003.04664*, 2020.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PMLR, 2021.
- Ren, Z., Dong, K., Zhou, Y., Liu, Q., and Peng, J. Exploration via hindsight goal generation. *Advances in Neural Information Processing Systems*, 32, 2019.
- Schaul, T., Horgan, D., Gregor, K., and Silver, D. Universal value function approximators. In *International conference on machine learning*, pp. 1312–1320. PMLR, 2015.
- Sigaud, O., Baldassarre, G., Colas, C., Doncieux, S., Duro, R., Oudeyer, P.-Y., Perrin-Gilbert, N., and Santucci, V. G. A definition of open-ended learning problems for goal-conditioned agents. *arXiv preprint arXiv:2311.00344*, 2023.

- Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., Casas, D. d. L., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Tirinzoni, A., Touati, A., Farebrother, J., Guzek, M., Kanervisto, A., Xu, Y., Lazaric, A., and Pirotta, M. Zero-shot whole-body humanoid control via behavioral foundation models. *arXiv preprint arXiv:2504.11054*, 2025.
- Touati, A. and Ollivier, Y. Learning one representation to optimize all rewards. *Advances in Neural Information Processing Systems*, 34:13–23, 2021.
- Touati, A., Rapin, J., and Ollivier, Y. Does zero-shot reinforcement learning exist? *arXiv preprint arXiv:2209.14935*, 2022.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.
- Wulfmeier, M., Ondruska, P., and Posner, I. Deep maximum entropy inverse reinforcement learning. *arXiv preprint arXiv:1507.04888*, 2015. URL <https://doi.org/10.48550/arxiv.1507.04888>.
- Yarats, D., Brandfonbrener, D., Liu, H., Laskin, M., Abbeel, P., Lazaric, A., and Pinto, L. Don’t change the algorithm, change the data: Exploratory data for offline reinforcement learning. *arXiv preprint arXiv:2201.13425*, 2022.
- Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.-H., Arenas, M. G., Chiang, H.-T. L., Erez, T., Hasenclever, L., Humplik, J., et al. Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647*, 2023.
- Ziebart, B. D., Maas, A. L., Bagnell, J. A., Dey, A. K., et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pp. 1433–1438. Chicago, IL, USA, 2008.

A. Proof of Proposition 4.1

Proof. For a fixed task z , the variance of the returns is determined by the projection of the task vector onto the directions of variance in the behavioral space:

$$\text{Var}_\pi(J(\pi, z)) = \text{Var}_\pi((\psi^\pi)^\top z) \quad (18)$$

$$= \mathbb{E}_\pi \left[((\psi^\pi - \bar{\psi})^\top z)^2 \right] \quad (19)$$

$$= \mathbb{E}_\pi [z^\top (\psi^\pi - \bar{\psi})(\psi^\pi - \bar{\psi})^\top z] \quad (20)$$

$$= z^\top (\mathbb{E}_\pi [(\psi^\pi - \bar{\psi})(\psi^\pi - \bar{\psi})^\top]) z \quad (21)$$

$$= z^\top \Sigma_\Psi z. \quad (22)$$

Now consider the expectation over the task distribution z . Using $\text{Tr}(ABC) = \text{Tr}(BCA)$ and linearity of expectation:

$$\mathbb{E}_z [z^\top \Sigma_\Psi z] = \mathbb{E}_z [\text{Tr}(z^\top \Sigma_\Psi z)] \quad (23)$$

$$= \mathbb{E}_z [\text{Tr}(\Sigma_\Psi z z^\top)] \quad (24)$$

$$= \text{Tr}(\Sigma_\Psi \mathbb{E}_z [z z^\top]). \quad (25)$$

For a uniform random vector on the unit hypersphere $\mathbb{S}^{d-1}$,

$$\mathbb{E}_z [z z^\top] = \frac{1}{d} I_d.$$

Substituting yields

$$\mathbb{E}_z [\text{Var}_\pi(J(\pi, z))] = \frac{1}{d} \text{Tr}(\Sigma_\Psi) = \frac{1}{d} \sum_{i=1}^d \lambda_i. \quad (26)$$

The goal is to prove that this converges to 0 as the dimension d grows.

The set of all possible behaviors (feature occupancies) is constrained by the environment's dynamics (e.g., gravity, collision limits, transition probabilities). We assume that the MDP $\mathcal{M}$ is fixed and independent of the representation dimension d : the complexity of the environment does not grow as d increases. Consequently, the variance of the valid behaviors is concentrated in a limited number of effective dimensions. As we increase d , we add new dimensions that capture finer details of the behavioral space. However, because the total information content of the MDP is finite, the variance contributed by these additional dimensions must eventually vanish.

Formally, this implies that the sequence of eigenvalues $\{\lambda_i\}_{i=1}^\infty$ satisfy:

$$\lim_{i \rightarrow \infty} \lambda_i = 0. \quad (27)$$

This condition allows the total variance to grow, provided the incremental variance added by each new dimension diminishes. Using **Cauchy's Limit Theorem**:

$$\lim_{i \rightarrow \infty} \lambda_i = 0 \implies \lim_{d \rightarrow \infty} \frac{1}{d} \sum_{i=1}^d \lambda_i = 0. \quad (28)$$

This completes the proof. □

B. Method details

B.1. Architecture

We maintain consistent network architectures across all evaluated methods:

- The successor feature network $\phi(s)$ and backward representation $B(s)$ are implemented as feedforward networks with three 256-unit hidden layers that output L_2 -normalized embeddings. We enforce the orthonormality of learned features through a regularization term.
- Our policy employs a TD3-based architecture (Fujimoto et al., 2018) with task-conditioned networks: the actor $\pi(s, z)$ uses parallel preprocessing streams (one for state features, another for task-conditioned states) with LayerNorm-Tanh initialization followed by ReLU activations, converging through a shared trunk network with Tanh output activation and truncated normal exploration ($\sigma = 0.2$). The critic uses twin Q-networks with analogous two-stream preprocessing of (s, a, z) tuples for stable value estimation.

B.2. Training the State Encoder

The state encoder $\phi(s)$ is trained using one of several representation learning objectives (see Appendix D).

The training process for FB representations is slightly different from the other methods. Although the FB objective simultaneously learns a backward embedding $B(s)$ and a policy π , we do not use them directly. Instead, to maintain a unified framework across all experimental baselines, we define the final state encoder as $\phi(s) = \mathbb{E}[BB^\top]^{-1}B(s)$ and train a new conditional policy from scratch using the reward $R_z(s) = \phi(s)^\top z$. We found that this approach does not hurt the performance of FB. Rather, it slightly improves it in the experiments, probably because the policy is trained on a fixed state representation, rather than a non-stationary one as in standard FB.

B.3. Training the Policy

During policy training, instead of sampling 2048 tuples (s_t, a_t, s_{t+1}, z) , we first sample 32 task vectors and then sample 64 transitions (s_t, a_t, s_{t+1}) per task. This was more efficient in our experiments, as sampling multiple transitions per reward function allows the critic to more rapidly capture the structure of each reward landscape.

B.4. Hyperparameters

Table 1. Hyperparameters for Dataset, Representation Learning, and Policy Training

Category	Parameter	Value
Dataset	Total Transitions	10^7
	Number of Trajectories	10^4
	Trajectory Length	10^3
	Replay Buffer Size	5×10^6 (10×10^6 for maze)
SF Training	Gradient Updates	10^5
	Batch Size	1024
	Learning Rate	10^{-4}
	Target Update Rate (τ)	0.01
	Discount Factor (γ)	0.99
	Orthonormality Weight	1
	Optimizer	Adam
FB Training	Gradient Updates	2×10^6
	Batch Size	1024
	Learning Rate	10^{-4}
	Target Momentum	0.99
	Orthonormality Weight	1
	Optimizer	Adam
Policy (TD3)	Gradient Updates	10^6
	Batch Size	2048
	Actor Learning Rate	10^{-4}
	Critic Learning Rate	10^{-4}
	Target Update Rate (τ)	0.01
	Discount Factor (γ)	0.99
	Exploration Noise (σ)	0.2
	Policy Smoothing Truncation	0.3
	Optimizer	Adam

C. Environment details

This section details the physical dimensions and reward objectives for the continuous control environments used in our evaluations. All the environments considered in this paper are based on the DeepMind Control Suite (Tassa et al., 2018):

Walker Walker is a planar bipedal robot. The 24-dimensional state vector contains joint positions and velocities. Actions are 6-dimensional vectors. We evaluate performance across four distinct tasks at test time:

- **Stand:** Reward is a weighted combination of terms that encourage an upright torso orientation and maintain a minimum torso height.
- **Walk:** Reward includes a component that encourages a minimum forward velocity, capped at 4 m/s.
- **Run:** Reward includes a component that encourages a higher minimum forward velocity, capped at 10 m/s.
- **Flip:** Reward includes a component that encourages the agent to achieve a minimum angular momentum.

Cheetah The Cheetah is a planar biped designed. The 17-dimensional state vector contains positions and velocities. Actions are 6-dimensional vectors. We consider four velocity-based tasks at test time:

- **Walk:** Reward is linearly proportional to forward velocity, capped at 2 m/s.
- **Run:** Reward is linearly proportional to forward velocity, capped at 10 m/s.

- **Walk Backward:** Reward is linearly proportional to backward velocity, capped at -2 m/s.
- **Run Backward:** Reward is linearly proportional to backward velocity, capped at -10 m/s.

Quadruped The Quadruped is a four-legged ant-like robot navigating in 3D space. The state and action spaces are 78-dimensional and 12-dimensional, respectively. We evaluate the following four tasks at test time:

- **Stand:** Reward encourages maintaining an upright torso posture.
- **Walk:** Reward includes a component that encourages a minimum forward torso velocity, capped at 0.5 m/s.
- **Run:** Reward includes a component that encourages higher forward torso velocities, capped at 5 m/s.
- **Jump:** Reward is proportional to the vertical displacement of the torso, it encourages the agent to maximize its peak height relative to the ground.

Across all environments, we utilize datasets from the ExORL (Yarats et al., 2022) benchmark collected via Random Network Distillation (RND) (Burda et al., 2018). Each dataset consists of 10^7 total transitions, composed of 10^4 trajectories with a length of 10^3 steps each.

D. Baselines details

This section provides objective functions for all baseline representation learning methods used in our experiments:

- **Autoencoder (AEnc)** (Chen & Guo, 2023) learns a compact representation $\phi(s)$ by minimizing the reconstruction error of the input state through an encoder-decoder architecture, capturing the most salient features of the state space. Here, ϕ is the encoder $\phi(s)$ and f is the decoder.

$$\min_{f, \phi} \mathbb{E}_{s \sim \mathcal{D}} [(f(\phi(s_t)) - s)^2].$$

- **Transition Model (Trans)** learns representations by training a model to predict the next state s_{t+1} given the current state-action pair (s_t, a_t) , thereby encoding the environment’s local dynamics into the latent space. Here, ϕ is the encoder $\phi(s)$ and f is the latent transition predictor.

$$\min_{f, \phi} \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [(f(\phi(s_t), a_t) - s_{t+1})^2].$$

- **Lower-Rank Approximation of the transition probability (LRA_P)** (Touati et al., 2022) factorizes the one-step transition probability $P(s'|s, a)$ into a low-rank product $f(s, a)^\top \phi(s')$, effectively performing a spectral decomposition of the transition operator. Here, f and ϕ represent the left and right singular vectors of the transition matrix.

$$\min_{f, \phi} \frac{1}{2} \mathbb{E}_{(s_t, a_t) \sim \mathcal{D}} \left[\mathbb{E}_{s' \sim \mathcal{D}} [((f(s_t, a_t)^\top \phi(s'))^2)] - \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [f(s_t, a_t)^\top \phi(s_{t+1})] \right].$$

- **Lower-Rank Approximation of the successor measure (LRA_SR)** (Touati et al., 2022) learns a low-rank factorization of the successor measure $M(s, s')$ using temporal difference learning. Here, f and ϕ are the learned factors that represent the state and its temporally extended features.

$$\min_{f, \phi} \mathbb{E}_{(s_t, s_{t+1}) \sim \mathcal{D}} \left[(f(s_t)^\top \phi(s') - \gamma f(s_{t+1})^\top \phi(s'))^2 \right] - 2 \mathbb{E}_{(s_t, s_{t+1}) \sim \mathcal{D}} [f(s_t)^\top \phi(s_{t+1})].$$

- **Bootstrap Your Own Latent (BYOL)** (Grill et al., 2020) learns a latent space by predicting the next state representation from the current state-action pair using a predictor and a target network. Here, ϕ is the encoder and ψ is the online predictor.

$$\min_{\phi, \psi} \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [\|\psi(\phi(s_t), a_t) - \text{sg}(\phi(s_{t+1}))\|_2^2].$$

- **Bootstrap Your Own Latent with Bidirectional Prediction (BYOL- γ)** (Lawson et al., 2025) captures long-range temporal consistency by predicting future states at horizons sampled geometrically. Here, ψ_f is the forward predictor towards future states and ψ_b is the backward predictor towards past states.

$$\min_{\phi, \psi_f, \psi_b} \mathbb{E}_{(s_t, a) \sim \mathcal{D}} \left[\mathbb{E}_{k \sim \text{Geom}(1-\gamma)} [\|\psi_f(\phi(s_t), a) - \text{sg}(\phi(s_{t+k}))\|_2^2 + \|\psi_b(\text{sg}(\phi(s_{t+k}))) - \phi(s_t)\|_2^2] \right].$$

- **Forward-Backward Representations (FB)** (Touati & Ollivier, 2021) jointly learns forward $F(s, a, z)$ and backward $B(s)$ representations to estimate successor measures for any arbitrary policy π_z . Here, F predicts future behavior under a goal/task z , while B encodes state-specific features.

$$\min_{F, B} \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[(F(s_t, a_t, z)^\top B(s') - \gamma F(s_{t+1}, \pi_z(s_{t+1}), z)^\top B(s'))^2 \right] - 2 \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [F(s_t, a_t, z)^\top B(s_{t+1})].$$

E. Additional Experiments: Ablation Study

This section presents a series of ablation studies designed to isolate and validate the key design choices underlying BTD-sampling. We investigate (1) whether performance gains arise from continuous density modeling rather than from heuristic task extraction, (2) empirically validate the signal-dilution phenomenon predicted in Proposition 4.1, (3) analyze the sensitivity to the number of mixture components, and (4) visualize the structure of the learned task space.

Table 2. Ablation of task sampling strategies. Zero-shot test performance comparison of BT-D against heuristic baselines. BT-D-sampling outperforms (Subtraj) sampling by interpolating the task space, while (full traj) sampling fails, as full trajectories are dominated by failures where the useful signal is minority.

Sampling Strategy	Cheetah	Walker	Quadruped
Uniform	464 $\pm$ 60	560 $\pm$ 42	634 $\pm$ 124
Full trajectory	482 $\pm$ 37	488 $\pm$ 11	421 $\pm$ 30
Subtrajectory	512 $\pm$ 24	579 $\pm$ 25	678 $\pm$ 43
BT-D-sampling (ours)	579 $\pm$ 25	611 $\pm$ 23	704 $\pm$ 64

E.1. Comparison to Heuristic Task Sampling

This experiment investigates whether the gains observed with BT-D-sampling stem from its continuous density modeling or merely from extracting tasks from the offline dataset through p_{data} . We compare BT-D-sampling with two heuristic baselines: (i) **Subtrajectory sampling**, where tasks are directly sampled from p_{data} ; (ii) **Full trajectory sampling**, where task vectors are computed from entire trajectories.

Results: BT-D-sampling consistently outperforms **Subtrajectory sampling** across all benchmarks (Table 2). Although both rely on p_{data} , BT-D enables interpolation between tasks and yields more diverse tasks.

In contrast, **Full trajectory sampling** performs poorly, particularly on Quadruped. As full trajectories contain multiple behaviors over time, they can produce task vectors where meaningful primitive behaviors are drowned into more noisy signal. In environments where recovery from failure is difficult, such as Quadruped, the resulting task vectors are dominated by failure and provide poor training reward functions.

E.2. Empirical Validation of Signal Dilution

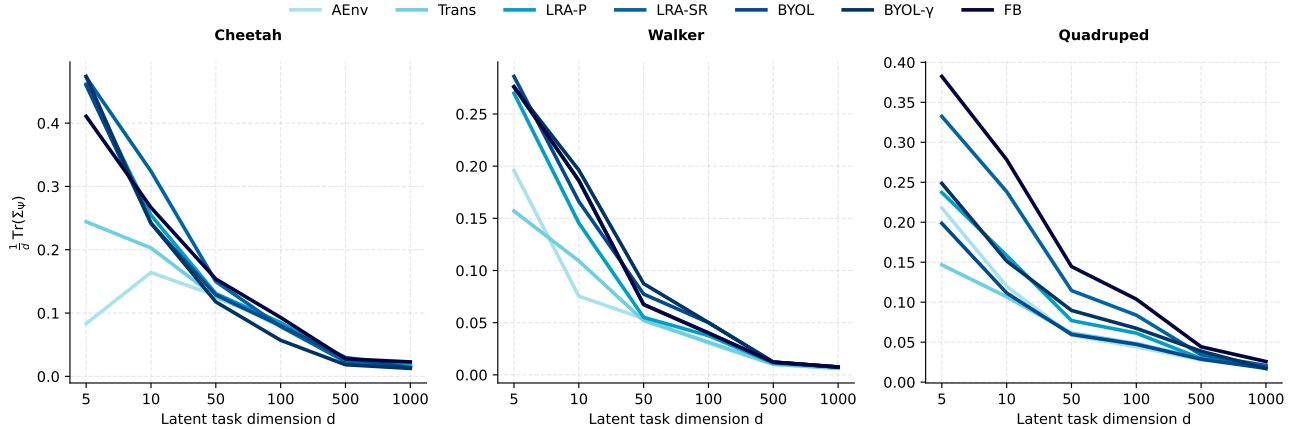


Figure 5. Signal dilution under increasing feature dimension. The normalized trace of the empirical feature-occupancy covariance $(1/d)\text{Tr}(\Sigma_{\Psi})$ decays rapidly as the feature dimension d increases across environments and baselines.

Proposition 4.1 characterizes the decay of the expected return variance when the tasks are uniformly sampled. The theoretical quantity Ψ represents the set of all possible feature occupancies under the MDP, which is not directly observable. We therefore construct an empirical counterpart $\tilde{\Psi}$ from the offline dataset by sampling 100,000 random subtrajectories τ and computing their discounted feature occupancies:

$$\tilde{\Psi} = \{\tilde{\psi}^{\tau} \mid \tau \in \mathcal{D}_{sub}\}, \quad \tilde{\psi}^{\tau} = \sum_{t=0}^{|\tau|} \gamma^t \phi(s_t). \quad (29)$$

For each environment, each baseline method, and each feature dimension $d \in \{5, 10, 50, 100, 500, 1000\}$, we train the corresponding state representation $\phi(s)$. We then compute $\tilde{\Psi}$ and its covariance $\Sigma_{\tilde{\Psi}}$, and report the upper bound quantity $(1/d)\text{Tr}(\Sigma_{\tilde{\Psi}})$.

Results: Figure 5 shows a rapid exponential decay of $(1/d)\text{Tr}(\Sigma_{\tilde{\Psi}})$ as d increases, confirming the signal-dilution effect predicted by Proposition 4.1. This confirms that the vanishing variance is not an artifact of a specific representation or environment, but a fundamental consequence of uniform sampling.

E.3. Effect of number of components in the GMM

This ablation study examines the impact of the number of Gaussian Mixture Model (GMM) components on the test performance across the benchmark environments and all the baselines, across a range of 5 to 200 GMM components.

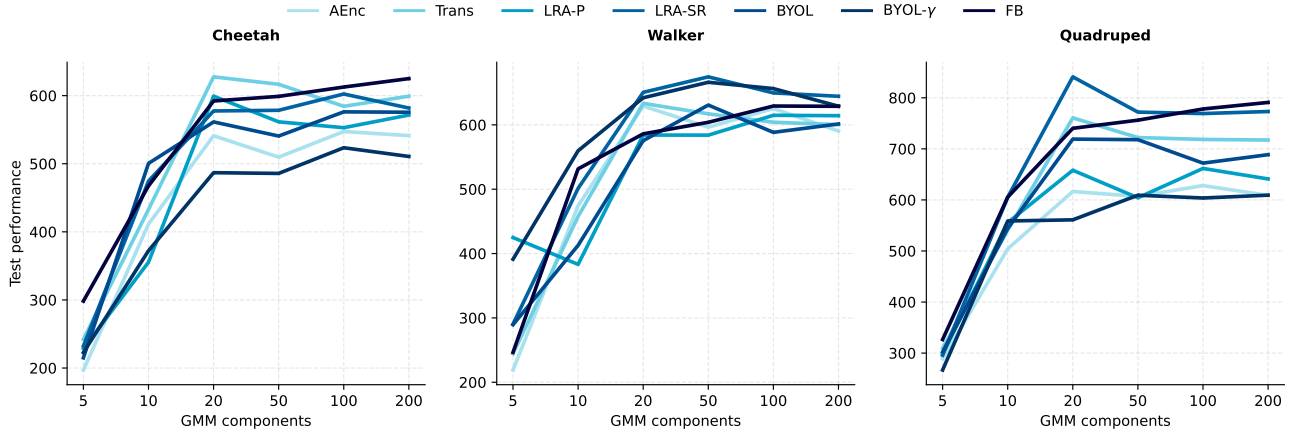


Figure 6. Effect of the number of GMM components on test performance. Performance improves rapidly with more components up to about 20, after which gains generally plateau across environments and baselines.

Results: Across all environments and baseline methods, we observe a consistent trend: performance increases rapidly until approximately 20 components, after which gains diminish. This indicates that the behavioral task distribution can be effectively captured with a relatively small number of components (20), and the exact choice beyond this point is not critical.

E.4. Effect of the Number of Subtrajectories N_τ

This ablation study examines the impact of the number of subtrajectories N_τ sampled from the dataset to fit the GMM, across a range from 10 to 10^6 . We use FB (BTD) as the baseline method.

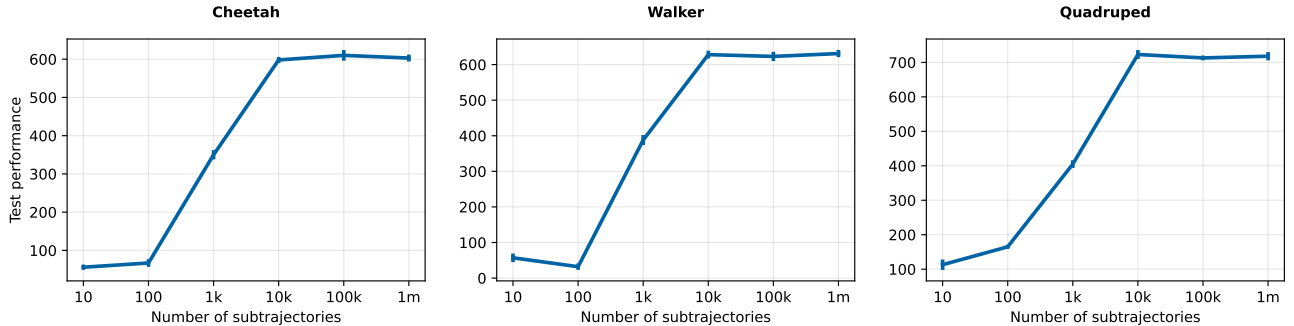


Figure 7. Effect of the number of subtrajectories N_τ on test performance. Performance improves rapidly with more subtrajectories up to about 10k, after which gains plateau across environments, indicating sufficient coverage of the behavioral task distribution.

Results: Across all three environments, performance improves consistently as N_τ increases, then saturates around $N_\tau = 10,000$, beyond which additional subtrajectories yield negligible gains. Small values of N_τ produce poor estimates of the behavioral task distribution and consequently low performance, while $N_\tau = 10,000$ provides sufficient coverage of the behavioral space. In practice, we use $N_\tau = 10,000$ as the default, as it offers a good trade-off between estimation quality and computational cost.

E.5. Visualization of the Task Space

We use UMAP projections to visualize the alignment between the empirical task distribution ($z \sim p_{data}$), extracted directly from the offline dataset, and the standard uniform prior ($z \sim \text{Unif}(S^{d-1})$). By comparing these distributions, we evaluate whether the uniform task sampling effectively captures the behavioral task distribution.

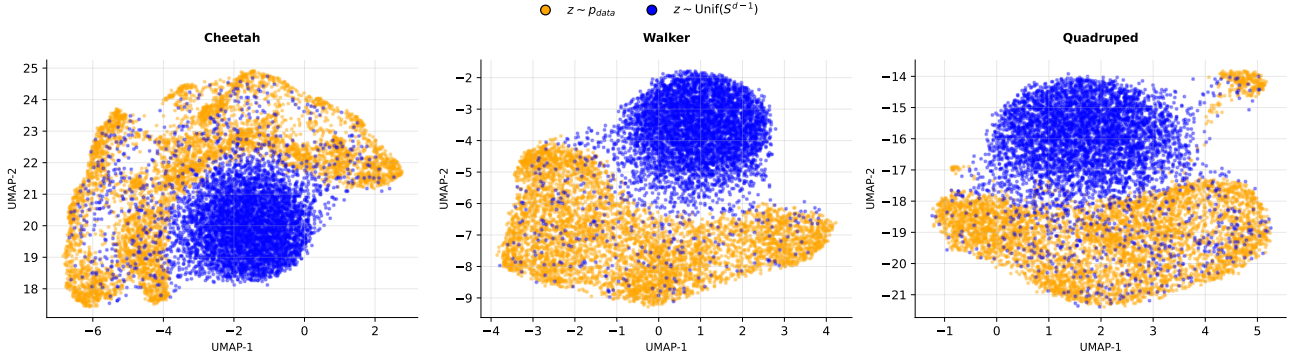


Figure 8. UMAP visualization of task distributions. Comparison between empirical task vectors $z \sim p_{data}$ and uniformly sampled task vectors $z \sim \text{Unif}(S^{d-1})$, showing limited overlap.

Results: The visualizations in Figure 8 reveal a distributional shift between the uniform prior and the data-driven task vectors. While the uniform prior theoretically spans the entire hypersphere, our results show only marginal overlap with the empirical task manifold. This shows that the behavioral task vectors reside in a small region of the hypersphere that cannot be efficiently recovered using uniform sampling.